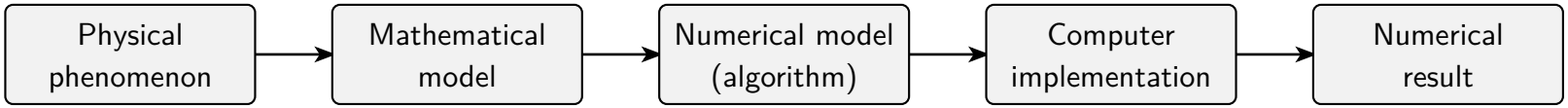
Chapter 7

Representation of real numbers

Numerical analysis - from phenomenon to result

Numerical analysis: methods to solve mathematical problems **with a computer**



Physical phenomenon: the real system, e.g. magnetic field around a power cable

Mathematical model: equations describing the phenomenon, e.g. Maxwell equations + constitutive laws + boundary conditions

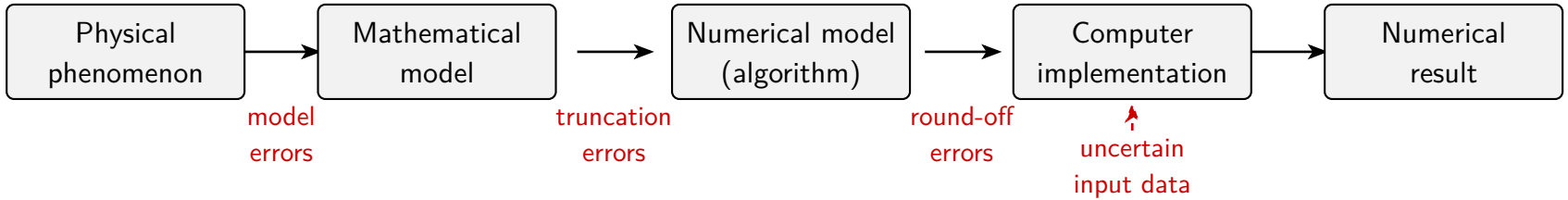
Numerical model: an **algorithm**, i.e. a complete and unambiguous set of procedures leading to the solution, using only operations a computer can do ($+$, $-$, $\times$, $\div$, finitely many times)

Computer implementation: algorithm written in a programming language (MATLAB) and executed

Numerical result: a finite set of numbers approximating the solution, e.g. $\boldsymbol{B}$ at the nodes of a grid

Numerical analysis - sources of error

Question: which errors does each step introduce?



Model errors: simplifying hypotheses, e.g. linear materials, 2D geometry

Truncation errors: continuous operations (derivatives, integrals, series) replaced by finite ones, e.g. $\mathrm{d}f/\mathrm{d}x \approx (f(x+h)-f(x))/h$

Name from series cut after a few terms: $e^x = 1 + x + x^2/2 + \ldots \approx 1 + x$

Round-off errors: real numbers stored with finitely many digits, e.g. $1/3 \approx 0.3333$ (this chapter)

Input data errors: data known approximately, e.g. measured σ, μ, dimensions

> Truncation and round-off come from the numerics; **all** errors **propagate** to the result

Warm-up (in class)

Done in class, in MATLAB: three results that look wrong

Section `%% warm-up` of `01_floating_point/floating_point_numbers.m`

Explained at the end of the chapter, in *Back to the warm-up*

Warm-up

Positional systems

Machine numbers and rounding

Fixed point

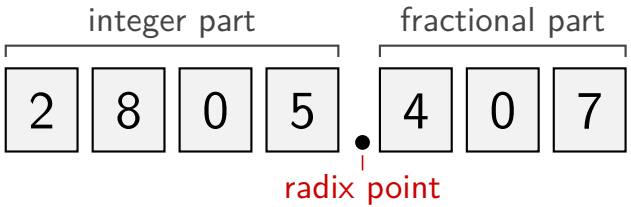
Floating point

Back to the warm-up

Number representation - positional systems

Question: how to write any real number with a finite set of symbols, the **digits**?

Split the number into an **integer part** and a **fractional part**, separated by the **radix point**:



Base $\beta \geq 2$: β digits $0, 1, \ldots, \beta - 1$ (base 10: $0, \ldots, 9$; base 2: $0, 1$)

The **position** of a digit fixes its **weight**, the power of β that multiplies it: in 2805.407, the 8 is worth $8 \cdot 10^2$, the 7 is worth $7 \cdot 10^{-3}$

Number representation - integer and fractional part

Integer part, digits $0 \leq a_k \leq \beta - 1$, with $a_n \neq 0$ (no leading zeros):

$$(a_n a_{n-1} \ldots a_1 a_0)_\beta = a_n \beta^n + a_{n-1} \beta^{n-1} + \ldots + a_1 \beta^1 + a_0 \beta^0 = \sum_{k=0}^{n} a_k \beta^k$$

Fractional part, digits $0 \leq b_k \leq \beta - 1$, possibly infinitely many:

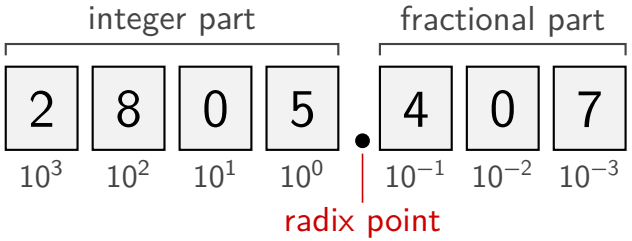
$$(0.b_1 b_2 \ldots)_\beta = b_1 \beta^{-1} + b_2 \beta^{-2} + \ldots = \sum_{k=1}^{\infty} b_k \beta^{-k}$$

Real number: the two parts, joined by the radix point:

$$(a_n \ldots a_1 a_0 . b_1 b_2 \ldots)_\beta = \underbrace{\sum_{k=0}^{n} a_k \beta^k}_{\text{integer part}} + \underbrace{\sum_{k=1}^{\infty} b_k \beta^{-k}}_{\text{fractional part}}$$

Number representation - example

Example $(2805.407)_{10}$: base $\beta = 10$, four integer digits, three fractional digits



$$(2805.407)_{10} = \underbrace{2 \cdot 10^3 + 8 \cdot 10^2 + 0 \cdot 10^1 + 5 \cdot 10^0}_{\text{integer part}} + \underbrace{4 \cdot 10^{-1} + 0 \cdot 10^{-2} + 7 \cdot 10^{-3}}_{\text{fractional part}}$$

Representation of real numbers

Warm-up

Positional systems

Machine numbers and rounding

Fixed point

Floating point

Back to the warm-up

Machine numbers - a finite set

Question: can a computer store every real number?

No: finite memory ⇒ only a **finite** set of numbers, the **machine numbers**

Two problems to solve:

1. **Which** numbers? A grid on the real line, built with a fixed number of digits. Two grids in this chapter: **fixed point** and **floating point**

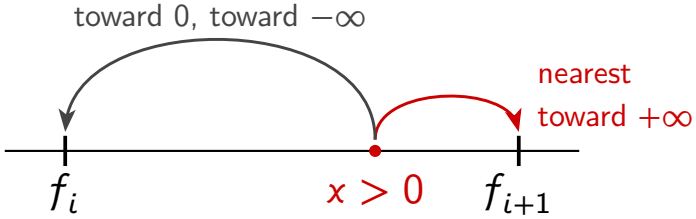
2. The real numbers are **dense**: between two machine numbers lie infinitely many reals ⇒ a **rounding rule** is needed, whatever the grid

Machine numbers - rounding

Assume a real number x between two consecutive machine numbers, $f_i < x < f_{i+1}$

Rounding: x is replaced by a machine number $\mathrm{fl}(x)$, chosen by a rule:

- **toward** 0 (chopping): the closest between x and 0
- **toward** $+\infty$: the closest between x and $+\infty$
- **toward** $-\infty$: the closest between x and $-\infty$
- **to nearest**: the closest one (default on computers)



Example grid with spacing 0.001 (three decimal digits):

	toward 0	toward $+\infty$	toward $-\infty$	nearest
$\mathrm{fl}(2/3)$	0.666	0.667	0.666	0.667
$\mathrm{fl}(-2/3)$	−0.666	−0.666	−0.667	−0.667

Representation of real numbers

Warm-up

Positional systems

Machine numbers and rounding

Fixed point

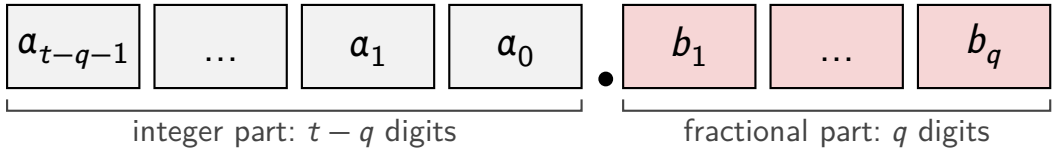
Floating point

Back to the warm-up

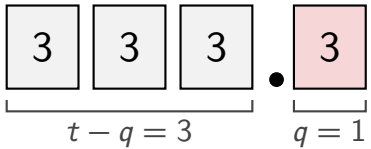
Fixed point representation - idea

Idea: a fixed number t of digits, with the radix point in a **fixed** position

q digits for the fractional part, $t - q$ for the integer part:



Example, $\beta = 10$, $t = 4$, $q = 1$: three integer digits and one fractional digit, e.g. 333.3



Fixed point representation - the set $\boldsymbol{X}$

Fixed point numbers with base β, t digits, q of them fractional:

$$\boldsymbol{X}(\beta, t, q) = \left\{ x = \text{sign}(x) \left[\underbrace{\sum_{k=0}^{t-q-1} a_k \beta^k}_{\text{integer part: } t-q \text{ digits}} + \underbrace{\sum_{k=1}^{q} b_k \beta^{-k}}_{\text{fractional part: } q \text{ digits}} \right] \right\}$$

digits $0 \leq a_k, b_k \leq \beta - 1$; leading zeros allowed, e.g. 000.1

Positive elements: each of the t digits takes β values $\rightarrow \beta^t$ strings, minus the all-zero one: $\beta^t - 1$

Largest: all digits $\beta - 1$ (e.g. 999.9 in $\boldsymbol{X}(10, 4, 1)$)

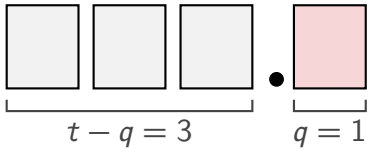
1. Without the radix point: one less than 1 followed by t zeros, $\beta^t - 1$ $(9999 = 10^4 - 1)$
2. Radix point q digits from the right = division by β^q: $x_M = (\beta^t - 1)\beta^{-q}$ $(999.9 = 9999/10)$

Smallest positive: only the last digit is 1: $x_m = \beta^{-q}$

Spacing: one unit of the last digit, β^{-q}, the **same** everywhere

Fixed point representation - example

Example $\boldsymbol{X}(10, 4, 1)$: $\beta = 10$, $t = 4$, $q = 1$



$$
\begin{aligned}
\text{largest:} \quad & x_M = (\beta^t - 1)\beta^{-q} = \left(10^4 - 1\right) \cdot 10^{-1} = 999.9 \\
\text{smallest positive:} \quad & x_m = \beta^{-q} = 10^{-1} = 000.1 \\
\text{spacing:} \quad & \Delta = \beta^{-q} = 10^{-1} = 0.1 \\
\text{positive elements:} \quad & \beta^t - 1 = 10^4 - 1 = 9999
\end{aligned}
$$

Fixed point - absolute vs relative error

Question: how large is the round-off error in $\boldsymbol{X}(10, 4, 1)$?

Absolute error: $E(x) = x - \mathrm{fl}(x)$ **Relative** error: $e(x) = |E(x)/x|$

x	$\mathrm{fl}(x)$	$E(x)$	$e(x)$
$x_1 = 10^3/3 = 333.\overline{3}$	333.3	$0.0\overline{3}$	10^{-4}
$x_2 = 1/3 = 0.\overline{3}$	000.3	$0.0\overline{3}$	10^{-1}

Constant spacing $\Rightarrow$ same **absolute** error; the **relative** error grows as $x \to x_m$: four correct digits for x_1, one for x_2

- **Pros**: fast and efficient arithmetic $\to$ graphics, computer games, DSP
- **Cons**: limited range and precision $\to$ rarely used in scientific computing

---

Warm-up
Positional systems
Machine numbers and rounding
Fixed point
Floating point
Back to the warm-up

Floating point representation - idea

Idea: keep the digits and move the radix point with a power of the base: the point **floats**

$$2805.407 = 280.5407 \cdot 10^1 = 2.805407 \cdot 10^3 = 0.2805407 \cdot 10^4 = \ldots$$

Same number, many forms: the **exponent** records where the radix point is

Normalized form: exactly one non-zero digit before the radix point

$$2805.407 = 2.805407 \cdot 10^3$$

Goal, compared with fixed point: same number of digits, but **wider range** and **bounded relative error**

Floating point representation - normalized form

Any real number $x \neq 0$, in base β, can be written as

$$x = \underbrace{\text{sign}(x)}_{\text{sign } \pm 1} \cdot \underbrace{\left[\sum_{k=0}^{\infty} d_k \beta^{-k} \right]}_{\text{mantissa } m} \cdot \underbrace{\beta^p}_{\text{exponential part}}$$

$p \in \mathbb{Z}$: **exponent**; d_k: digits of the mantissa

Example

$$2805.407 = \underbrace{\left(2 \cdot 10^0 + 8 \cdot 10^{-1} + 0 \cdot 10^{-2} + 5 \cdot 10^{-3} + 4 \cdot 10^{-4} + 0 \cdot 10^{-5} + 7 \cdot 10^{-6}\right)}_{m=2.805407} \cdot \underbrace{10^3}_{p=3}$$

Floating point representation - digits

Every digit, as in any positional system: $0 \leq d_k \leq \beta - 1$

Special rule for the **first** digit: $d_0 \neq 0$ (**normalization**)

otherwise the form is not unique: $2.805407 \cdot 10^3 = 0.2805407 \cdot 10^4 = 0.02805407 \cdot 10^5 = \ldots$

No infinite tail of $(\beta - 1)$ digits

again two forms of the same number: $9.\overline{9} \cdot 10^0 = 1.0 \cdot 10^1$

Consequence for the mantissa:

$$1 \leq m < \beta$$

$d_0 \geq 1 \Rightarrow m \geq 1$; $m = \beta$ only with all digits $\beta - 1$ (e.g. $9.\overline{9} = 10$), excluded

Floating point representation - the set $\boldsymbol{F}$

Normalized form: **infinitely many** digits, **any** exponent $p \in \mathbb{Z}$: a computer must limit both

Keep only t digits of the mantissa, and bound the exponent: $L \leq p \leq U$

Floating point numbers with base β, t digits, exponent between L and U:

$$\boldsymbol{F}(\beta, t, L, U) = \{0\} \cup \left\{ x = \text{sign}(x) \left[\sum_{k=0}^{t-1} d_k \beta^{-k} \right] \beta^p, \quad L \leq p \leq U \right\}$$

0 is added on its own: it has no normalized form ($d_0 \neq 0$)

Same digit rules as before: $d_0 \neq 0$, $0 \leq d_k \leq \beta - 1$

Floating point representation - example

Example $\boldsymbol{F}(10, 3, 0, 9)$: $\beta = 10$, $t = 3$, $L = 0$, $U = 9$

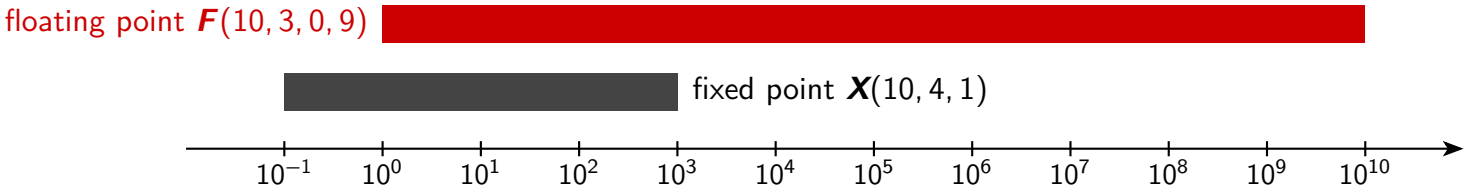
Largest mantissa times largest power, and smallest times smallest:

$$\text{largest:} \quad x_M = \underbrace{9.99}_{\text{all digits } \beta-1=9} \cdot \underbrace{10^9}_{p=U} = 9.99 \cdot 10^9$$

$$\text{smallest positive:} \quad x_m = \underbrace{1.00}_{d_0=1,\ \text{others } 0} \cdot \underbrace{10^0}_{p=L} = 1$$

as a sum of digits: $9.99 = 9 \cdot 10^0 + 9 \cdot 10^{-1} + 9 \cdot 10^{-2} = 10 - 0.01$; in general, largest mantissa $= \beta - \beta^{1-t}$

$\boldsymbol{F}(10, 3, 0, 9)$ vs $\boldsymbol{X}(10, 4, 1)$: both use **4 digits** (3 for the mantissa + 1 for the exponent)



Ten orders of magnitude instead of four

Floating point - spacing

Question: how far apart are consecutive elements of $\boldsymbol{F}(10, 3, 0, 9)$?

mantissa m	p	elements of $\boldsymbol{F}$: $m \cdot 10^p$	spacing Δ
$1.00, 1.01, \ldots, 9.99$	0	$1.00, 1.01, 1.02, \ldots, 9.99$	10^{-2}
$1.00, 1.01, \ldots, 9.99$	1	$10.0, 10.1, 10.2, \ldots, 99.9$	10^{-1}
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$1.00, 1.01, \ldots, 9.99$	9	$1.00 \cdot 10^9, 1.01 \cdot 10^9, \ldots, 9.99 \cdot 10^9$	10^7

Same mantissas in every row: only the exponent p changes, and moves the radix point

Mantissa spacing β^{1-t} (one unit of the last digit), scaled by the exponential part:

$\Delta = \beta^{1-t}\beta^p$: **not** constant, it grows with $|x|$

Floating point - absolute vs relative error

Question: as for fixed point, how large is the round-off error in $\boldsymbol{F}(10, 3, 0, 9)$?

x	$\mathrm{fl}(x)$	$E(x)$	$e(x)$
$x_1 = 10/3 = 3.\overline{3}$	3.33	$3.\overline{3} \cdot 10^{-3}$	10^{-3}
$x_2 = 10^3/3 = 333.\overline{3}$	$3.33 \cdot 10^2$	$3.\overline{3} \cdot 10^{-1}$	10^{-3}
$x_3 = 10^9/3$	$3.33 \cdot 10^8$	$3.\overline{3} \cdot 10^5$	10^{-3}

Absolute error grows with $|x|$, as the spacing; the relative error does **not** depend on p: same mantissa, same $e(x)$

fixed point: the opposite, constant $E(x)$ and growing $e(x)$

> We will show: $e(x) < \beta^{1-t}$ for every x in the range of $\boldsymbol{F}$: **bounded** relative error

Floating point - relative round-off error

Hypotheses: x in the range of $\boldsymbol{F}$ ($L \leq p \leq U$), rounding toward zero (chopping)

Chopping: $\mathrm{fl}(x)$ keeps the first t digits of x, with the same sign and exponent

$$x = \mathrm{sign}(x)\left[\sum_{k=0}^{\infty} d_k \beta^{-k}\right]\beta^p, \quad \mathrm{fl}(x) = \mathrm{sign}(x)\left[\sum_{k=0}^{t-1} d_k \beta^{-k}\right]\beta^p$$

The rounding mode enters here: with rounding to nearest, the last kept digit may change ($2/3 \to 0.667$)

Substituting into $e(x) = |x - \mathrm{fl}(x)| \,/\, |x|$: sign($x$) and β^p cancel out

$$e(x) = \frac{\cancel{|\mathrm{sign}(x)|}\left[\sum_{k=0}^{\infty} d_k \beta^{-k} - \sum_{k=0}^{t-1} d_k \beta^{-k}\right]\cancel{\beta^p}|}{\cancel{|\mathrm{sign}(x)|}\left[\sum_{k=0}^{\infty} d_k \beta^{-k}\right]\cancel{\beta^p}|} = \frac{\left|\sum_{k=0}^{\infty} d_k \beta^{-k} - \sum_{k=0}^{t-1} d_k \beta^{-k}\right|}{\sum_{k=0}^{\infty} d_k \beta^{-k}}$$

Floating point - splitting the sum

Splitting the infinite sum at $k = t$:

$$\sum_{k=0}^{\infty} d_k \beta^{-k} = \underbrace{\sum_{k=0}^{t-1} d_k \beta^{-k}}_{\text{first } t \text{ digits: kept by fl}(x)} + \underbrace{\sum_{k=t}^{\infty} d_k \beta^{-k}}_{\text{digits discarded}}$$

Substituting into the numerator of $e(x)$:

$$\left| \sum_{k=0}^{\infty} d_k \beta^{-k} - \sum_{k=0}^{t-1} d_k \beta^{-k} \right| = \left| \cancel{\sum_{k=0}^{t-1} d_k \beta^{-k}} + \sum_{k=t}^{\infty} d_k \beta^{-k} - \cancel{\sum_{k=0}^{t-1} d_k \beta^{-k}} \right| = \sum_{k=t}^{\infty} d_k \beta^{-k}$$

no absolute value needed at the end: all terms are ≥ 0

$$e(x) = \frac{\overbrace{\sum_{k=t}^{\infty} d_k \beta^{-k}}^{\text{discarded digits}}}{\underbrace{\sum_{k=0}^{\infty} d_k \beta^{-k}}_{\text{mantissa } m}}$$

Floating point - bound on the round-off error (1/2)

Denominator: it is the mantissa, so $m \geq 1$; dividing by $m \geq 1$ cannot increase the fraction:

$$e(x) = \frac{\sum_{k=t}^{\infty} d_k \beta^{-k}}{m} \leq \sum_{k=t}^{\infty} d_k \beta^{-k}$$

Numerator: writing the terms out and collecting β^{-t}:

$$\sum_{k=t}^{\infty} d_k \beta^{-k} = d_t \beta^{-t} + d_{t+1} \beta^{-(t+1)} + d_{t+2} \beta^{-(t+2)} + \ldots$$

$$\downarrow \quad \beta^{-(t+j)} = \beta^{-t} \beta^{-j}$$

$$= \beta^{-t} \left(d_t + d_{t+1} \beta^{-1} + d_{t+2} \beta^{-2} + \ldots \right)$$

Floating point - bound on the round-off error (2/2)

We found:

$$e(x) \leq \beta^{-t}\left(d_t + d_{t+1}\beta^{-1} + d_{t+2}\beta^{-2} + \ldots\right)$$

The bracket has the form of a mantissa: digits $0 \leq d_k \leq \beta - 1$, no infinite tail of $(\beta - 1)$ digits. As for the mantissa $(1 \leq m < \beta)$, it is $< \beta$

here $d_t = 0$ is possible, so the bracket may be < 1: only the upper bound is used

$$e(x) < \beta^{-t} \cdot \beta = \beta^{1-t}$$

The bound does not depend on x: neither on its exponent nor on its digits

Floating point - machine precision

Machine precision ε: the maximum relative round-off error

$$\varepsilon = k\beta^{1-t}, \quad k = \begin{cases} 1/2 & \text{rounding to nearest} \\ 1 & \text{other rounding modes} \end{cases}$$

Rounding toward zero: $k = 1$, just proved

Rounding to nearest: x is never farther than **half a spacing** from $\mathrm{fl}(x) \Rightarrow k = 1/2$

ε depends only on β, t and the rounding mode, **not** on x

e.g. $\boldsymbol{F}(10, 3, 0, 9)$, rounding to nearest: $\varepsilon = 1/2 \cdot 10^{-2} = 0.5\%$

Floating point - round-off model

So what? We know $e(x) \leq \varepsilon$: how is the stored number $\mathrm{fl}(x)$ related to the exact x?

Relative error without the absolute value (**signed**): $\delta = (\mathrm{fl}(x) - x)/x, \quad |\delta| \leq \varepsilon$

Solving for $\mathrm{fl}(x)$:

$$\delta = \frac{\mathrm{fl}(x) - x}{x}$$

$$\Big\downarrow \text{ multiplying by } x, \text{ adding } x$$

$$\mathrm{fl}(x) = x + x\delta = x(1 + \delta)$$

$$\mathrm{fl}(x) = x(1 + \delta), \quad |\delta| \leq \varepsilon$$

A machine number is the real one times a factor between $1 - \varepsilon$ and $1 + \varepsilon$

Floating point - the binary system

Computer logic is based on two-valued pulses (ON/OFF) $\Rightarrow$ base $\beta = 2$, digits (**bits**) 0 or 1

Same arithmetic rules as in base 10 (tables on the right)

$1 + 1 = (10)_2 = 2$: write 0, carry 1

Normalized binary: the first digit is non-zero $\Rightarrow$ $d_0 = 1$ **always**

addition

+	0	1
0	0	1
1	1	10

multiplication

×	0	1
0	0	0
1	0	1

Hidden bit: d_0 is not stored $\rightarrow$ one extra significant digit for free

Floating point - a toy binary system

Example $\boldsymbol{F}(2, 3, -1, 2)$: base 2, $t = 3$, $-1 \leq p \leq 2$. Small enough to list **all** its elements

---

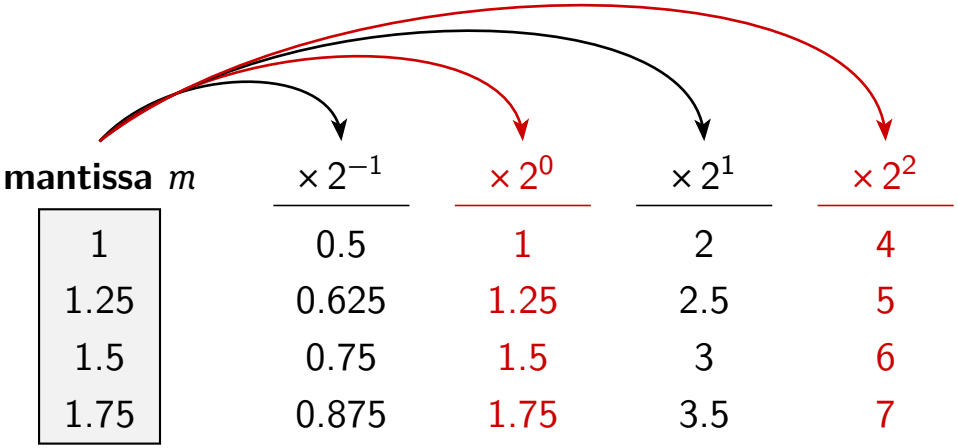
Mantissas: $d_0 = 1$ always, $d_1, d_2 \in \{0, 1\}$; spacing $2^{-2} = 0.25$:

$$(1.00)_2 = 1 \cdot 2^0 + 0 \cdot 2^{-1} + 0 \cdot 2^{-2} = 1$$
$$(1.01)_2 = 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} = 1.25$$
$$(1.10)_2 = 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} = 1.5$$
$$(1.11)_2 = 1 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} = 1.75$$

Exponential parts: 2^p, $p = -1, 0, 1, 2$: $0.5, 1, 2, 4$

Elements: every mantissa times every exponential part, $4 \times 4 = 16$ positive numbers ($+$ negatives and 0)

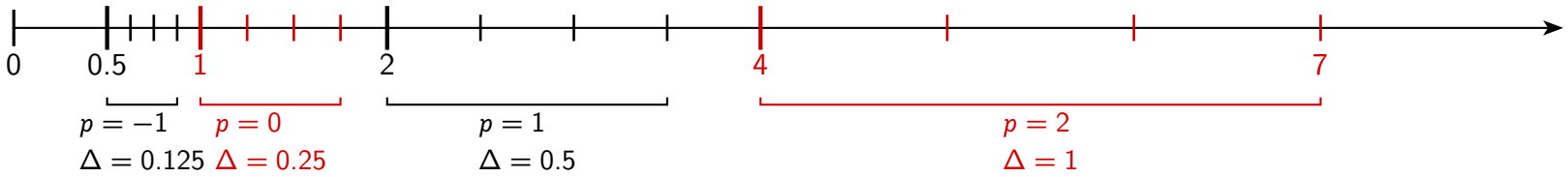
Floating point - toy system: all elements



mantissa m	$\times 2^{-1}$	$\times 2^{0}$	$\times 2^{1}$	$\times 2^{2}$
1	0.5	1	2	4
1.25	0.625	1.25	2.5	5
1.5	0.75	1.5	3	6
1.75	0.875	1.75	3.5	7

Each column: the four mantissas times one exponential part; spacing within a column $\Delta = 2^{-2} \cdot 2^{p}$, i.e. $\Delta = \beta^{1-t}\beta^{p}$

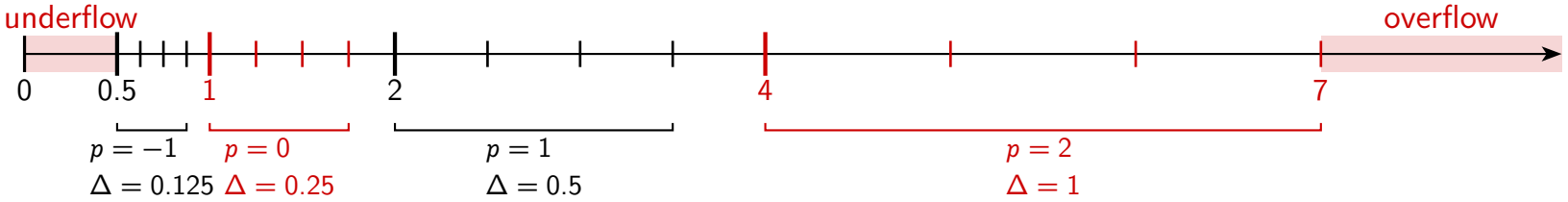
Floating point - toy system on the real line



Spacing Δ constant within each group, **doubling** from one group to the next

Next group: $\Delta' = 2\Delta$, $x' = 2x \Rightarrow \Delta'/x' = \Delta/x$: same **relative** spacing

Floating point - toy system: underflow and overflow



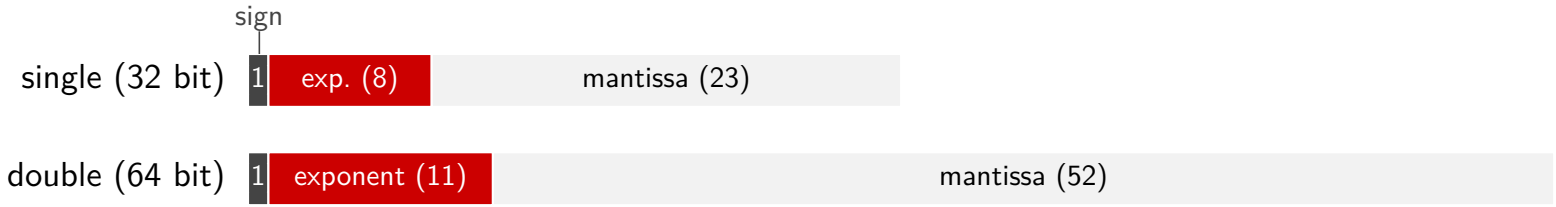
Underflow: $0 < |x| < 0.5$, below the smallest element; the gap $(0, 0.5)$ is much wider than the first spacing

Overflow: $|x| > 7$, above the largest element

$\varepsilon = k2^{1-3} = k \cdot 0.25$: relative error at most 12.5% (nearest), 25% (other rounding modes)

Floating point - the IEEE 754 standard

Real computers: the same idea as the toy system, with much longer fields



format	exponent p	t (with hidden bit)	$\varepsilon = k2^{1-t}$
single	$-126 \ldots 127$	24	$k \cdot 2^{-23} \approx k \cdot 1.19 \cdot 10^{-7}$
double	$-1022 \ldots 1023$	53	$k \cdot 2^{-52} \approx k \cdot 2.22 \cdot 10^{-16}$

MATLAB `eps` $= 2^{-52}$: distance from 1 to the next double ($k = 1$); rounding to nearest: $|\delta| \leq \texttt{eps}/2 \approx 1.1 \cdot 10^{-16}$

It also fixes rounding, arithmetic and special values (**subnormals**, `Inf`, `NaN`)

Used by all modern processors and languages (MATLAB, Python, C, ...)

Floating point - arithmetic

IEEE 754: each operation ($+$, $-$, $\times$, $\div$) on two machine numbers is computed **exactly**, then the result is rounded

Notation: x, y real numbers; a, $b \in \boldsymbol{F}$ machine numbers

Operation on machine numbers: one round-off, of the result

$$\mathrm{fl}(a+b) = (a+b)(1+\delta), \quad |\delta| \leq \varepsilon$$

Real data (e.g. $x = 0.1$, $y = 0.2$): first stored, then summed

$$\begin{aligned} \text{stored:} \quad & a = \mathrm{fl}(x) = x(1+\delta_1), \quad b = \mathrm{fl}(y) = y(1+\delta_2) \\ \text{summed:} \quad & \mathrm{fl}(a+b) = (x(1+\delta_1) + y(1+\delta_2))(1+\delta_3) \end{aligned}$$

three round-offs: two of the data, one of the result

Floating point - arithmetic, example

Example toy system, rounding to nearest: $a = -1.75$, $b = -0.875$, $c = 6$

$$\mathrm{fl}(\mathrm{fl}(a + b) + c)$$

$$a + b = -2.625 \rightarrow \mathrm{fl} = -2.5$$

$$-2.5 + 6 = 3.5 \rightarrow \mathrm{fl} = 3.5$$

3.5

$$\mathrm{fl}(\mathrm{fl}(a + c) + b)$$

$$a + c = 4.25 \rightarrow \mathrm{fl} = 4.0$$

$$4.0 - 0.875 = 3.125 \rightarrow \mathrm{fl} = 3.0$$

3.0

Floating point addition is not **associative**: the order of the sums changes the result

---

Warm-up

Positional systems

Machine numbers and rounding

Fixed point

Floating point

Back to the warm-up

Back to the warm-up - $0.1 + 0.2 \neq 0.3$

0.1 in base 2: $(0.0001100110011\ldots)_2$, infinitely many bits, like $1/3 = 0.\overline{3}$ in base 10 $\Rightarrow$ rounded: $\mathrm{fl}(0.1) = 0.1000000000000000055511\ldots$ (the same for 0.2 and 0.3)

The sum works on $\mathrm{fl}(0.1) + \mathrm{fl}(0.2)$, then rounds again: the result lands on the double **next to** $\mathrm{fl}(0.3)$

Their difference, $5.5511 \cdot 10^{-17}$, is exactly 2^{-54}: one spacing of the doubles in $[0.25, 0.5)$, $\Delta = \beta^{1-t}\beta^{p} = 2^{-52} \cdot 2^{-2}$

> Never compare floating point numbers with ==: use a tolerance
> `abs(a - b) <= tol*abs(b)` or `isapprox(a, b)` (MATLAB R2024b+)

Back to the warm-up - order of the sums, cancellation

W2: every partial sum is rounded, as in the toy system

$$\texttt{0.1 + 0.2} \to 0.30000000000000004 \qquad \texttt{0.2 + 0.3} \to 0.5$$
$$0.30000000000000004 + 0.3 \to 0.6000000000000001 \qquad 0.5 + 0.1 \to 0.6$$

W3: $\cos(10^{-8}) = 1 - 5 \cdot 10^{-17} + \ldots$, but the doubles near 1 are spaced by $\approx 10^{-16} \Rightarrow \mathrm{fl}(\cos x) = 1$, and $1 - \mathrm{fl}(\cos x) = 0$

All the significant digits are lost in the subtraction: **cancellation** (lab, step 3)

> MATLAB is not wrong: it works exactly as IEEE 754 prescribes, on the finite set $\boldsymbol{F}$

Double precision - significant digits

ε is **relative**: about 16 correct significant digits, whatever the magnitude of x

	x	fl(x) (double)	$E(x)$
$x_1 = 10^{20}/3$	$3.3333333333333333333... \cdot 10^{19}$	$3.3333333333333331968 \cdot 10^{19}$	$\approx 1.4 \cdot 10^{3}$
$x_2 = 10^{-20}/3$	$3.3333333333333333333... \cdot 10^{-21}$	$3.3333333333333332758... \cdot 10^{-21}$	$\approx 5.7 \cdot 10^{-38}$

Black: the first 16 significant digits, the same in x and fl(x); red: the wrong digits of fl(x)

Absolute errors 41 orders of magnitude apart, relative errors both below $\varepsilon \approx 1.1 \cdot 10^{-16}$

Representation of real numbers - take-home

- Computers work on a **finite** set $\boldsymbol{F}$: every real number is rounded
- Floating point: wide range and **bounded relative error**, $|\delta| \leq \varepsilon = k\beta^{1-t}$
- ε is **relative**: in double precision ($\varepsilon \approx 1.1 \cdot 10^{-16}$) about 16 significant digits, for large and small numbers alike
- Arithmetic is **not** exact: the order of operations matters, subtracting close numbers cancels digits, == is unreliable

Lab - Floating point numbers (in class)

Done in class, in MATLAB

Folder: `01_floating_point`

Goal: limits of double precision, lost digits, cancellation; plain script `floating_point_numbers.m` (sections `%%`)